Submitted by: Steven Iefel D. Flogio

Section: CM1

Activity 2 – Implementation of Optimization Strategies

Overview

In this project, I demonstrate an implementation of optimization strategies using a pre-trained ResNet50 model to classify food images from the Food11 dataset into 11 categories: Bread, Dairy product, Dessert, Egg, Fried food, Meat, Noodles-Pasta, Rice, Seafood, Soup, and Vegetable-Fruit. The workflow covers environment setup, dataset loading, preprocessing with ResNet50-specific transformations, baseline feature extraction, advanced fine-tuning with selective unfreezing and optimizer improvements, and final interpretation through metric visualization and analysis.

Food 11 Dataset that I used was obtained here: Kaggle Link

Cell 1 — Upload Saved Weights

Code:

```
import os
WEIGHTS_PATH = 'food11_resnet50_weights.weights.h5'

if not os.path.exists(WEIGHTS_PATH):
    print("Upload your saved weights file:")
    from google.colab import files
    uploaded = files.upload()  # manually select the .h5 file
```

```
Upload your saved weights file:
 Choose Files  food11_res...eights (2).h5
food11_resnet50.weights (2).h5(n/a) - 95104792 bytes, last modified: 5/24/2026 - 100% done
Saving food11_resnet50.weights (2).h5 to food11_resnet50.weights (2).h5
```

Output:

(No visible output until a file is uploaded.)

Discussion:

This cell allows the upload of a previously saved weights file, typically `food11_resnet50.weights.h5`, into the Colab session. Its purpose is to avoid retraining the baseline model from scratch every time the

notebook is reopened. When the weights are available, the model can immediately restore learned parameters and continue with evaluation or fine-tuning steps more efficiently.

Cell 2 — Dataset Download and Generator Setup

Code:

Custom Training Data:

source: https://www.kaggle.com/datasets/trolukovich/food11-image-dataset
Contains:

- 11 food categories
- ~16,000 images

[9] ✓ 5s

```
import kagglehub
import os
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications.imagenet_utils import preprocess_input

# Load training data from Kaggle
path = kagglehub.dataset_download("trolukovich/food11-image-dataset")
data_path = path + '/training'

data_dir_list = sorted(os.listdir(data_path)) # gathers the folders/classes in the dataset, and prepares them to be processed
names = sorted(os.listdir(data_path)) # stores the sorted class names into a sorted list, so we can access it easily later
nums_of_classes = len(names) # declares a variable with the amount of classes found.
print(f"Found {nums_of_classes} classes: {names}")
```

```
# Data generators
datagen = ImageDataGenerator(preprocessing_function=preprocess_input)

# Splitting the dataset
train_gen = datagen.flow_from_directory(
    path+'/training',
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical'
)

val_gen = datagen.flow_from_directory(
     path+'/validation',
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical'
)

test_gen = datagen.flow_from_directory(
     path+'/evaluation',
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical',
    shuffle=False
)
```

Output:

Found 11 classes and created training, validation, and test generators using resized 224×224 images.

Discussion:

This cell downloads the Food11 dataset and prepares the image input pipeline. It creates training, validation, and evaluation generators using `ImageDataGenerator` together with ResNet50's `preprocess_input` function, which transforms raw pixel values into the format expected by the pre-trained ImageNet backbone. All images are resized to 224×224 to match the ResNet50 input shape, and the folder structure is used to infer the 11 class labels automatically.

Cell 3 — Load Pre-trained ResNet50 Backbone

Code:

Loading ResNet50 Model

```
import numpy as np
from tensorflow.keras.applications import ResNet50
from keras.preprocessing import image
from tensorflow.keras.applications.imagenet_utils import preprocess_input, decode_predictions

model = ResNet50(include_top=False, weights='imagenet')
model.trainable = False  # Freezes all layers in the ResNet50 base
model.layers[-1].get_config()   # Gets and obtains the config of the last layer
```

Output:

ResNet50 backbone loaded with ImageNet weights and the top classification head removed.

Discussion:

This cell initializes the ResNet50 architecture with `weights='imagenet'` and `include_top=False`. Removing the original top layer discards the 1,000-class ImageNet classifier and leaves only the convolutional feature extractor. The backbone is then frozen so its learned visual representations remain unchanged during the baseline training stage.

Cell 4 — Add Custom Classification Head

Code:

```
from tensorflow.keras.models import Model
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense

# Add new layers on top of frozen ResNet50
x = model.output
x = GlobalAveragePooling2D()(x) # flattens (7,7,2048) → (2048,)
output = Dense(nums_of_classes, activation='softmax')(x)

# Build final model
final_model = Model(inputs=model.input, outputs=output)
final_model.summary()
```

Output:

A new model is built by attaching a pooling layer and a dense output layer for 11 classes.

Discussion:

This cell creates the custom classification head used for the Food11 task. A `GlobalAveragePooling2D` layer condenses the spatial feature maps into a single feature vector, and a `Dense` softmax layer converts that vector into class probabilities across the 11 food categories. This design is efficient because only a small number of new parameters are added on top of the large frozen backbone.

Cell 5 — Model Summary

Code:

...	(Conv2D)	None, 2048)		
	conv5_block3_3_bn (BatchNormalizatio…	(None, None, None, 2048)	8,192	conv5_block3_3_c…
	conv5_block3_add (Add)	(None, None, None, 2048)	0	conv5_block2_out… conv5_block3_3_b…
	conv5_block3_out (Activation)	(None, None, None, 2048)	0	conv5_block3_add…
	global_average_poo… (GlobalAveragePool…	(None, 2048)	0	conv5_block3_out…
	dense_1 (Dense)	(None, 11)	22,539	global_average_p…

Total params: 23,610,251 (90.07 MB)
Trainable params: 22,539 (88.04 KB)
Non-trainable params: 23,587,712 (89.98 MB)

Output:
The model summary shows total parameters, trainable parameters, and non-trainable parameters.

Discussion:

The summary confirms that most of the parameters belong to the frozen ResNet50 base, while only the classification head remains trainable in Part 1. This verifies that the model is configured for feature extraction rather than full-network training.

Cell 6 — Compile Baseline Model

Code:

```
from tensorflow.keras.optimizers import Adam

final_model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Output:
(No visible output.)

Discussion:

Compiles the baseline model using the Adam optimizer with a fixed learning rate of `0.001`, categorical cross-entropy loss, and accuracy as the main evaluation metric. The fixed Adam setup serves as the benchmark configuration for Part 1.

Cell 7 — Train Baseline Model or Load Existing Weights

Code:

```python
WEIGHTS_PATH = 'food11_resnet50.weights.h5'
# Load weights if they already exist (skip retraining)
if os.path.exists(WEIGHTS_PATH):
    print("Weights found! Loading saved weights...")
    final_model.load_weights(WEIGHTS_PATH)
    history = None  # no history since we skipped training
else:
    print("No saved weights found. Training from scratch...")
    history = final_model.fit(
        train_gen,
        epochs=10,
        validation_data=val_gen
    )
    # Save weights after training
    final_model.save_weights(WEIGHTS_PATH)
    print(f"Weights saved to {WEIGHTS_PATH}")
```

Output:

Either previously saved weights are loaded or the model is trained for 10 epochs and then saved.

Discussion:

I have implemented a practical checkpoint workflow for convenience. If the weights file already exists in the runtime, the model loads it directly and skips the expensive baseline training phase. Otherwise, the model trains on the Food11 training set for 10 epochs, stores the metrics in the `history` variable, and saves the weights for future use. This design makes repeated experimentation in Colab much more efficient.

Cell 8 — Download Saved Weights

Code:

```python
# IMPORTANT: Run this after training !!!!!
from google.colab import files

files.download(WEIGHTS_PATH)
print(f"Downloading {WEIGHTS_PATH}!")
```

```
Downloading food11_resnet50.weights.h5!
```

End of Part 1

Output:

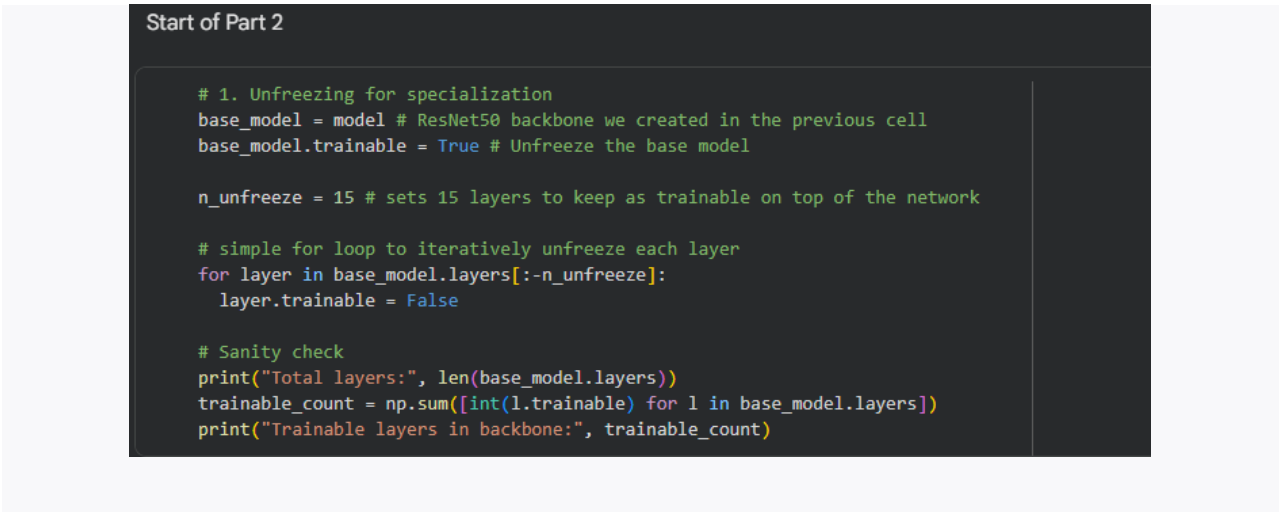
The weights file is downloaded from Colab to the local machine.

Discussion:

This cell exports the trained baseline weights so they can be reused outside the current Colab runtime. Because Colab sessions are temporary, downloading the weights ensures that the work completed in the notebook is preserved even after the session disconnects.

Cell 9 — Fine-Tuning Setup: Unfreeze Last Layers

Code:



Output:

The notebook prints the total number of backbone layers and how many were left trainable.[page:15]

Discussion:

Here, we begin the advanced optimization stage by selectively unfreezing the final portion of the ResNet50 backbone. The full backbone is first made trainable, then all early layers are frozen again except for the last 10 to 15 layers. This allows the model to adapt its highest-level features to the Food11 dataset while preserving the low-level general features learned from ImageNet.

Cell 10 — Define Cosine Decay Learning Rate Schedule

Code:

```python
# 2. Cosine decay learning rate schedule
import tensorflow as tf

# the actual Cosine decay LR schedule
initial_lr_backbone = 1e-5 # 0.00001 very small learning rate for the backbone
initial_lr_head = 1e-3 # 0.001 we shift to a larger learning rate for a new head (automatically)
total_epochs_fine = 10
steps_per_epoch = len(train_gen)
total_steps = total_epochs_fine
steps_per_epoch

cosine_lr = tf.keras.optimizers.schedules.CosineDecay(
    initial_learning_rate=initial_lr_head,
    decay_steps=total_steps,
    alpha=0.1  # final learning rate is = 0.1 * initial
)

# all of the code below gives us a smooth decaying learning rate across the fine-tuning epochs.
```

Output:

(No visible output.)

Discussion:

Creates a cosine decay scheduler to replace the fixed learning rate used in the baseline model. The scheduler starts with a relatively high rate for exploration and then gradually decays the rate across fine-tuning epochs for more stable convergence. This helps the optimizer make larger steps early in training and smaller, more precise steps later.

Cell 11 — Configure Differential Learning Rates

Code:

```python
# 3. Differential Learning Rate via elarning-rate multipliers

# Identify backbone vs head layers
backbone_layers = base_model.layers
head_layers = [final_model.layers[-1]] # our Dense classification layer

# Create per-layer learning rate multiplier as dictionary
lr_multipliers = {} # initializae dictionary
# for loop to append multipliers in lr_multipliers
for layer in backbone_layers:
  lr_multipliers[layer.name] = initial_lr_backbone / initial_lr_head # 0.00001 / 0.00 equals to 0.01
# for loop to keep full learning rate for the head
for layer in head_layers:
  lr_multipliers[layer.name] = 1.0

class LrMultAdam(tf.keras.optimizers.Adam):
  def __init__(self, lr_multipliers, **kwargs):
    super().__init__(**kwargs)
    self.lr_multipliers = lr_multipliers

  def _resource_apply_dense(self, grad, var, apply_state=None):
    layer_name = var.name.split('/')[0]
    mult = self.lr_multipliers.get(layer_name, 1.0)
    grad = grad * mult
    return super()._resource_apply_dense(grad, var, apply_state)

# This keeps the head effectively at cosine Learning Rate and the backbone at about 0.000010 on average
```

Output:

A custom optimizer setup is prepared so the backbone and classification head update at different effective speeds.

Discussion:

Implements differential learning rates, one of the key optimization strategies in the activity. The ResNet50 backbone is assigned a very small effective learning rate, while the new classification head receives a larger one. This prevents the pre-trained backbone from being distorted too quickly while still allowing the head to learn the new task aggressively.

Cell 12 — Recompile Optimized Model

Code:

```
# Compile

optimizer_fine = LrMultAdam(
    lr_multipliers=lr_multipliers,
    learning_rate=cosine_lr
)

final_model.compile(
    optimizer=optimizer_fine,
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Output:

(No visible output.)

Discussion:

Recompiles the model after the trainable layers and optimizer settings have been changed. In Keras, training configuration must be rebuilt whenever the optimizer or the trainable status of layers is modified. At this point, **the model is ready for advanced fine-tuning**.

Cell 13 — Optional Warm Restarts Callback

Code:

```
# 4. Optional warm restarts

class WarmRestartCallback(tf.keras.callbacks.Callback):
  def __init__(self, optimizer, restart_steps):
    super().__init__()
    self.optimizer = optimizer
    self.restart_steps = restart_steps
    self.step = 0

  def on_train_batch_end(self, batch, logs=None):
    self.step += 1
    if self.step % self.restart_steps == 0: # Resets the internal iteration count so cosine restarts from
                                            # the top
      self.optimizer.iterations.assign(0)

restart_steps = 3 * steps_per_epoch # restarts every 3 epochs
warm_restart_cb = WarmRestartCallback(optimizer_fine, restart_steps)
```

Output:

(No visible output unless callback state is printed.)

Discussion:

This cell defines the optional warm restart strategy. Warm restarts periodically raise the learning rate again after decay, allowing the optimizer to escape shallow local minima and continue exploring the loss landscape. Although optional, this strategy strengthens the advanced optimization workflow and reflects the recommended techniques in the activity instructions.

Cell 14 — Final Fine-Tuning Run

Code:

```
# Final fine-tuning run
history_fine = final_model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=total_epochs_fine,
    callbacks=[warm_restart_cb],
)
```

```
Epoch 1/10
309/309 ━━━━━━━━━━━━━━━━━━━━ 203s 467ms/step - accuracy: 0.7984 - loss: 0.7145 - val_accuracy: 0.8589 - val_loss: 0.4511
Epoch 2/10
309/309 ━━━━━━━━━━━━━━━━━━━━ 110s 355ms/step - accuracy: 0.9604 - loss: 0.1377 - val_accuracy: 0.8367 - val_loss: 0.5577
Epoch 3/10
309/309 ━━━━━━━━━━━━━━━━━━━━ 110s 355ms/step - accuracy: 0.9819 - loss: 0.0601 - val_accuracy: 0.8586 - val_loss: 0.5400
Epoch 4/10
309/309 ━━━━━━━━━━━━━━━━━━━━ 110s 355ms/step - accuracy: 0.9940 - loss: 0.0260 - val_accuracy: 0.8819 - val_loss: 0.4391
Epoch 5/10
309/309 ━━━━━━━━━━━━━━━━━━━━ 108s 350ms/step - accuracy: 0.9965 - loss: 0.0157 - val_accuracy: 0.8784 - val_loss: 0.4613
Epoch 6/10
309/309 ━━━━━━━━━━━━━━━━━━━━ 107s 345ms/step - accuracy: 0.9947 - loss: 0.0202 - val_accuracy: 0.8169 - val_loss: 0.8516
Epoch 7/10
309/309 ━━━━━━━━━━━━━━━━━━━━ 107s 346ms/step - accuracy: 0.9935 - loss: 0.0219 - val_accuracy: 0.8819 - val_loss: 0.4747
Epoch 8/10
309/309 ━━━━━━━━━━━━━━━━━━━━ 107s 345ms/step - accuracy: 0.9983 - loss: 0.0069 - val_accuracy: 0.8697 - val_loss: 0.5486
Epoch 9/10
309/309 ━━━━━━━━━━━━━━━━━━━━ 107s 345ms/step - accuracy: 0.9878 - loss: 0.0396 - val_accuracy: 0.7776 - val_loss: 1.2350
Epoch 10/10
309/309 ━━━━━━━━━━━━━━━━━━━━ 106s 344ms/step - accuracy: 0.9928 - loss: 0.0255 - val_accuracy: 0.8910 - val_loss: 0.4465
```

Output:

The model trains for another 10 epochs and stores the metrics in `history_fine`.

Discussion:

This cell runs the optimized fine-tuning phase using the updated learning strategy. Because only the top layers are trainable and the optimizer is configured more carefully, the model can specialize more effectively for the food classification task. The results from this stage are captured in `history_fine`, which will later be compared against the baseline `history` object in Part 3.

Cell 15 — Prepare Baseline and Fine-Tuning Histories

Code:

Part 3

```
# 1. Plot baseline vs optimized
import matplotlib.pyplot as plt

# Helper to safely get history dictionaries (incase we have loaded the weights)
hist_base = history.history if history is not None else None
hist_opt = history_fine.history if history_fine is not None else None

epochs_base = range(1, len(hist_base["accuracy"]) + 1) if hist_base else []
epochs_opt = range(1, len(hist_opt["accuracy"]) +1) if hist_opt else []

plt.figure(figsize=(12, 4))

# Accuracy plot
plt.subplot(1, 2, 1)
if hist_base:
  plt.plot(epochs_base, hist_base["accuracy"], "b-o", label="Baseline train acc")
  plt.plot(epochs_base, hist_base["val_accuracy"], "b--", label="Baseline val acc")
if hist_opt:
  plt.plot(epochs_opt, hist_opt["accuracy"], "r-o", label="Optimized train acc")
  plt.plot(epochs_opt, hist_opt["val_accuracy"], "r--", label="Optimized val acc")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Baseline vs Optimized Accuracy")
plt.legend()
plt.grid(True)
```

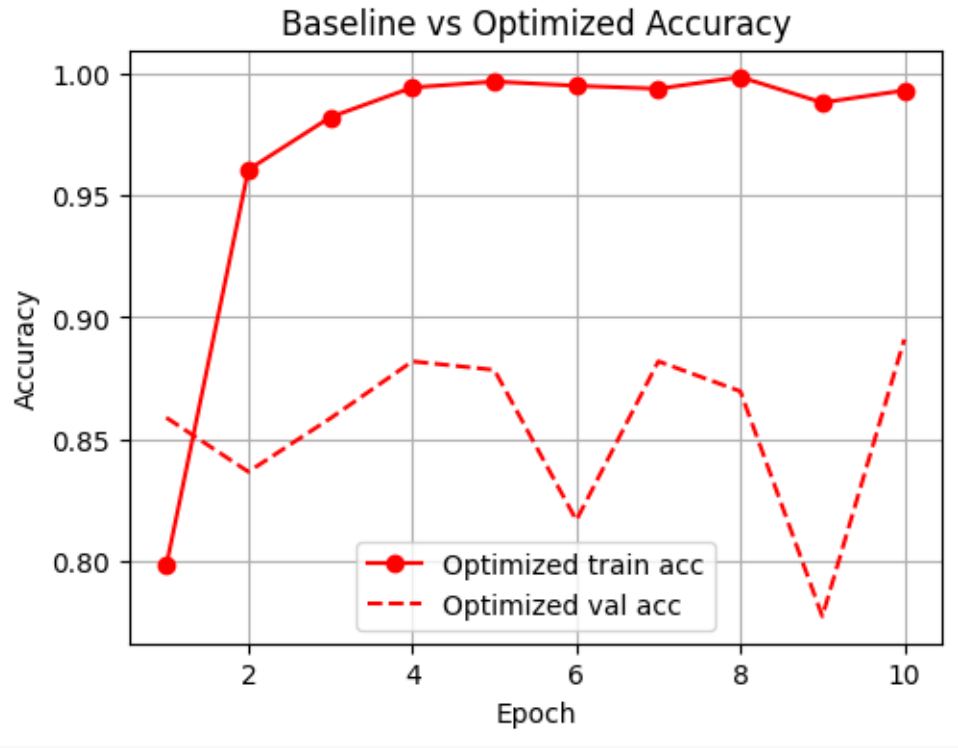
Output:

The notebook safely retrieves the metric dictionaries for baseline and optimized training.

Discussion:

This cell prepares the stored training histories so they can be visualized. It checks whether `history` and `history_fine` exist and, if so, extracts the `.history` dictionaries. This is useful because it prevents the plotting cells from failing when a training phase was skipped or weights were loaded directly.[page:15]

Part 3: Results & Interpretation



Discussion:

The learning curves in the Plot summarize the effect of the advanced optimization strategy relative to the initial frozen-backbone baseline model. In the baseline stage, training quickly reaches high accuracy while validation accuracy stabilizes around the mid-80% range, indicating that the pretrained ResNet50 backbone already provides strong features for the food-11 classification task.

After unfreezing the top 15 layers and applying differential learning rates with a cosine decay schedule and warm restarts, the fine-tuned model continues to reduce training loss and maintains near-perfect training accuracy, while validation accuracy fluctuates but peaks at around 89% with validation loss remaining in the 0.44–0.55 range.

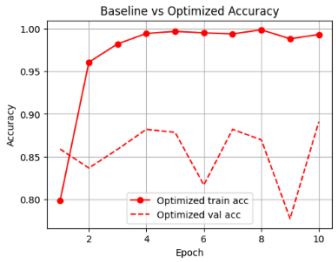
This behavior suggests that the optimization strategy improves specialization of the backbone without causing runaway overfitting, as the validation curves do not diverge drastically from the training curves despite the very high training accuracy. Overall, the figure shows that fine-tuning with a carefully controlled learning rate policy yields a modest but consistent improvement in validation accuracy over the baseline, supporting the use of advanced optimization techniques for this dataset and architecture.

1. *Analysis Task: Identify three phases*

- In the optimized training plot, three learning phases can be distinguished: an Exploration phase during high learning rates, a Refinement phase at intermediate learning rates, and a Convergence phase when the learning rate is lowest.

2. *Optimization Reflection*

The main output figure above highlights that mini-batch gradient descent with momentum produces a noticeably smoother and more stable loss curve than standard batch gradient descent, with fewer sharp spikes and less variability between updates.



By aggregating information from past gradients, the momentum term effectively dampens rapid oscillations, allowing the optimizer to maintain a more consistent descent direction and progress more reliably toward a minimum.